

Guida a GitHub

Dalle basi al workflow collaborativo completo

GitHub è la piattaforma dove pubblichi repository Git, collabori tramite pull request, tieni traccia del lavoro con issue e automatizzi con Actions.

Quick reference

Comando / Azione	Descrizione
<code>git clone URL</code>	Clona repository in locale
<code>git checkout -b feature/nome</code>	Crea e apre un branch
<code>git add .</code>	Aggiunge modifiche allo staging
<code>git commit -m "messaggio"</code>	Crea un commit
<code>git push -u origin feature/nome</code>	Pubblica branch su GitHub
Pull Request	Richiesta di merge su GitHub
<code>git pull --rebase origin main</code>	Allinea il branch locale
<code>git fetch --all --prune</code>	Aggiorna i riferimenti remoti
Issue	Traccia bug, task e richieste
Actions	Esegue CI/CD automatizzata

Indice

1	Cos'è GitHub	3
2	I concetti fondamentali	3
3	Setup iniziale	3
3.1	Configura Git	3
3.2	Genera una chiave SSH	3
3.3	Test connessione	4
4	Primo flusso di lavoro	4
4.1	Clona il repository	4
4.2	Crea un branch di lavoro	4

4.3	Lavora e salva	4
4.4	Pubblica su GitHub	4
5	Pull Request fatta bene	4
6	Aggiornare il branch senza caos	5
6.1	Allinea il locale	5
6.2	Porta le novità sul branch feature	5
7	Issue e organizzazione	5
8	README minimo consigliato	5
9	GitHub Actions in 60 secondi	6
10	Cheat sheet veloce	6
11	Errori comuni dei principianti	7
12	Workflow consigliato in team	7
13	Conclusione	7

1 Cos'è GitHub

GitHub è un servizio cloud basato su **Git**. Git è il motore di versionamento; GitHub aggiunge collaborazione e strumenti di team.

GitHub è utile quando:

- vuoi salvare e versionare codice in remoto;
- lavori in gruppo su branch separati;
- vuoi revisionare modifiche prima del merge;
- vuoi automatizzare test e deploy.

Idea pratica: usa Git in locale per sviluppare e GitHub per condividere, revisionare e integrare.

2 I concetti fondamentali

Repository

Contenitore del progetto (codice, storia commit, file di configurazione).

Branch

Linea di sviluppo separata, utile per feature o fix.

Commit

Snapshot coerente delle modifiche.

Pull Request (PR)

Richiesta di revisione e merge di un branch su un altro.

Issue

Ticket per bug, task, idee o discussioni operative.

Struttura mentale:

Repository → Branch → Commit → Pull Request

3 Setup iniziale

3.1 Configura Git

```
git config --global user.name "Nome Cognome"
git config --global user.email "mail@example.com"
```

3.2 Genera una chiave SSH

```
ssh-keygen -t ed25519 -C "mail@example.com"
cat ~/.ssh/id_ed25519.pub
```

Copia la chiave pubblica e aggiungila su GitHub in:

```
Settings > SSH and GPG keys
```

3.3 Test connessione

```
ssh -T git@github.com
```

4 Primo flusso di lavoro

4.1 Clona il repository

```
git clone git@github.com:org/progetto.git  
cd progetto
```

4.2 Crea un branch di lavoro

```
git checkout -b feature/login
```

4.3 Lavora e salva

```
git add .  
git commit -m "feat: aggiunge login base"
```

4.4 Pubblica su GitHub

```
git push -u origin feature/login
```

Poi, su GitHub, apri una **Pull Request** verso main.

5 Pull Request fatta bene

Una PR efficace riduce tempi di review e problemi in merge.

Elemento	Buona pratica
Titolo	Breve, orientato al risultato
Descrizione	Cosa cambia, perche, impatti
Scope	Piccolo e focalizzato
Checklist	Test, lint, documentazione
Reviewers	Assegna le persone giuste

Consiglio utile: prima di aprire la PR, aggiorna il branch con `main` per ridurre conflitti.

6 Aggiornare il branch senza caos

6.1 Allinea il locale

```
git checkout main
git pull origin main
```

6.2 Porta le novità sul branch feature

```
git checkout feature/login
git rebase main
```

Se ci sono conflitti:

1. risolvi i file;
2. esegui `git add <file>`;
3. continua con `git rebase -continue`.

Al termine, aggiorna il remoto:

```
git push -force-with-lease
```

7 Issue e organizzazione

Le **Issue** sono il punto di ingresso del lavoro.

Strumento	Uso pratico
Issue template	Standardizza bug report e task
Label	Classifica priorità e tipo lavoro
Milestone	Raggruppa task per release
Project board	Stato operativo (todo/doing/done)

Regola semplice: ogni PR dovrebbe riferirsi a una issue, così il contesto resta chiaro nel tempo.

8 README minimo consigliato

Un buon repository ha almeno:

- descrizione breve del progetto;
- prerequisiti;
- istruzioni di installazione e avvio;
- esempi d'uso;
- sezione contributi.

```
# Nome progetto
Descrizione breve
## Setup
## Uso
## Contribuire
```

9 GitHub Actions in 60 secondi

GitHub Actions serve per CI/CD: test automatici, lint, build e deploy.

File tipico:

```
.github/workflows/ci.yml
```

Schema base:

- trigger su push e pull_request;
- job con checkout codice;
- install dipendenze;
- esecuzione test/lint.

10 Cheat sheet veloce

Comando / Azione	Significato
<code>git clone URL</code>	Clona repository
<code>git status</code>	Mostra stato working tree
<code>git checkout -b nome</code>	Nuovo branch
<code>git add .</code>	Prepara modifiche per commit
<code>git commit -m "msg"</code>	Salva snapshot
<code>git push -u origin nome</code>	Pubblica branch
<code>git pull -rebase origin main</code>	Aggiorna branch locale
<code>git fetch -all -prune</code>	Pulisce riferimenti remoti vecchi
<code>git log -oneline -graph</code>	Cronologia compatta
Open Pull Request	Chiede review e merge
Merge Pull Request	Integra branch in target
Close Issue	Chiude task completato

11 Errori comuni dei principianti

1. Commit troppo grandi

Rendono difficile capire, revieware e fare rollback.

2. Lavorare direttamente su main

Meglio usare branch dedicati per ogni cambiamento.

3. Messaggi commit vaghi

Evita "fix" o "update": scrivi cosa e perché.

4. Aprire PR senza contesto

Descrivi sempre obiettivo, test fatti e impatti.

12 Workflow consigliato in team

1. Apri issue con obiettivo chiaro.
2. Crea branch da `main`.
3. Fai commit piccoli e descrittivi.
4. Pubblica branch e apri PR.
5. Ricevi review e applica feedback.
6. Esegui merge e chiudi issue.

13 Conclusione

GitHub diventa potente quando usi un flusso ordinato: branch puliti, PR piccole, review serie e automazioni CI.

Da ricordare:

- separa sempre il lavoro in branch;
- apri PR leggibili e testate;
- usa issue per tracciare decisioni e task;
- automatizza il controllo qualità con Actions.